

Day 4 — Loops Basics in Python

Detailed Seminar Notes | Syntax • Rules • Examples • Dry Run • Viva Points • Practice

A **loop** is used when we want to execute a block of code **again and again** without writing the same statement multiple times.

Instead of writing `print("Hello")` 10 times, we can use a loop to do it automatically. Loops are one of the most important topics in Python because they help in **repetition, automation, and problem solving**.

1) What is a Loop?

A loop is a control structure that repeatedly runs a block of code as long as a condition is true or for a fixed number of times.

- **Repetition:** perform the same task multiple times.
- **Less code:** avoids writing the same statement again and again.
- **Easy updates:** if logic changes, update one block instead of many lines.
- **Useful in programs:** printing tables, summing numbers, checking values in a list, patterns, menus, games, and more.

2) Why do we need loops?

Without loop	With loop
<pre>print("Python") print("Python") print("Python") print("Python") print("Python")</pre>	<pre>for i in range(5): print("Python")</pre>

3) Types of loops in Python

- **for loop** — used when we know how many times to repeat, or when we want to iterate through a sequence like a string, list, tuple, or range.
- **while loop** — used when repetition depends on a condition and we do not always know in advance how many times the loop will run.

4) for Loop

A **for loop** is used to iterate over a sequence or a range of values.

General syntax:

```
for variable in sequence:
    statement(s)
```

Important points:

- The word **for** is a Python keyword.
- **variable** stores the current value during each repetition.
- **sequence** can be a **range**, string, list, tuple, set, etc.

- The statements inside the loop must be **indented**.
- Each time the loop repeats, the variable gets a new value.

Example 1: Print numbers from 1 to 5

```
for i in range(1, 6):
    print(i)
```

Output: 1 2 3 4 5

Explanation: *range(1, 6)* gives 1, 2, 3, 4, 5. The value 6 is not included. So the loop runs 5 times.

Example 2: Print a word 3 times

```
for i in range(3):
    print("Hello")
```

Output: Hello Hello Hello

When we write *range(3)*, Python gives values 0, 1, 2. That means the loop runs 3 times.

Example 3: Loop through a string

```
for ch in "PYTHON":
    print(ch)
```

This prints each character one by one: P, Y, T, H, O, N.

5) range() function

The **range()** function is commonly used with a **for loop**. It generates a sequence of numbers.

Forms of range():

Syntax	Meaning	Example	Output values
<code>range(stop)</code>	Starts from 0 and goes up to stop-1	<code>range(5)</code>	0, 1, 2, 3, 4
<code>range(start, stop)</code>	Starts from start and goes up to stop-1	<code>range(2, 6)</code>	2, 3, 4, 5
<code>range(start, stop, step)</code>	Moves by step	<code>range(2, 10, 2)</code>	2, 4, 6, 8

Examples using range()

```
for i in range(5):
    print(i)          # 0 1 2 3 4

for i in range(2, 6):
    print(i)          # 2 3 4 5

for i in range(2, 11, 2):
    print(i)          # 2 4 6 8 10
```

6) while Loop

A **while loop** repeats a block of code **as long as a condition is True**.

General syntax:

```
while condition:
    statement(s)
```

Important points:

- The condition is checked **before** every repetition.

- If the condition is True, the loop body executes.
- If the condition becomes False, the loop stops.
- Usually we update a variable inside the loop; otherwise it may run forever.

Example 1: Print numbers 1 to 5 using while

```
i = 1
while i <= 5:
    print(i)
    i = i + 1
```

Output: 1 2 3 4 5

Explanation: Initially $i = 1$. The loop prints i and increases it by 1 each time. When i becomes 6, the condition $i \leq 5$ becomes False, so the loop stops.

Example 2: Print a word 4 times

```
count = 1
while count <= 4:
    print("Python")
    count += 1
```

7) Dry Run of a while loop

Dry run means tracing the program step by step to understand how variables change.

```
i = 1
while i <= 3:
    print(i)
    i += 1
```

Iteration	Value of i before checking	Condition $i \leq 3$	Printed value	New value of i
1	1	True	1	2
2	2	True	2	3
3	3	True	3	4
4	4	False	-	Loop stops

8) Rules to remember while using loops

- **Indentation is compulsory** in Python. The loop body must be indented.
- In a **while loop**, make sure the variable changes inside the loop. Otherwise you may get an **infinite loop**.
- In **range(start, stop)**, the **stop value is not included**.
- Choose **for loop** when the number of repetitions is known or when you are traversing a sequence.
- Choose **while loop** when repetition depends on a condition.
- Use meaningful variable names like i , $count$, num , depending on the problem.

9) Infinite Loop

An **infinite loop** is a loop that never stops because its condition never becomes False.

```
i = 1
while i <= 5:
    print(i)
```

This is an infinite loop because i is never updated. So $i \leq 5$ remains True forever.

Correct version:

```
i = 1
while i <= 5:
    print(i)
    i += 1
```

10) Difference between for loop and while loop

for loop	while loop
Used when number of repetitions is known or when iteration sequence depends on a condition.	Used when repetition depends on a condition.
Often used with range(), list, string, tuple, etc.	Often used with counters, menu-driven programs, and unknown repetition
Initialization and update are usually handled by Python <code>range()</code> or <code>range(start, stop, step)</code> .	Usually done manually.
Example: print 1 to 10	Example: keep asking until password is correct

11) Common beginner mistakes

- Forgetting the **colon :** after *for* or *while*.
- Wrong indentation inside the loop.
- Using `range(1, 5)` and expecting it to print 1 to 5. It actually prints 1 to 4.
- Forgetting to update the variable in a while loop.
- Writing the update statement outside the loop by mistake.
- Confusing `=` (assignment) with `==` (comparison) in conditions.

12) Important loop programs for beginners

Program 1: Print numbers from 1 to 10

```
for i in range(1, 11):
    print(i)
```

Program 2: Print even numbers from 2 to 20

```
for i in range(2, 21, 2):
    print(i)
```

Program 3: Print multiplication table of 5

```
n = 5
for i in range(1, 11):
    print(n, "x", i, "=", n * i)
```

Program 4: Find sum of first 5 natural numbers

```
sum = 0
for i in range(1, 6):
    sum = sum + i
print("Sum =", sum)
```

Output: Sum = 15

Program 5: Reverse counting from 10 to 1

```
for i in range(10, 0, -1):
    print(i)
```

13) Practice questions

- Print numbers from 1 to 20 using a for loop.
- Print numbers from 10 to 1 using a while loop.
- Print all odd numbers from 1 to 25.
- Print the multiplication table of any number entered by the user.
- Find the sum of numbers from 1 to 100.
- Count how many digits are there in a number.
- Find factorial of a number using a loop.
- Print a star pattern using nested loops later when you study nested loops.

14) Viva / seminar questions

- What is a loop in Python?
- Why do we use loops?
- What are the types of loops in Python?
- What is the difference between for loop and while loop?
- What is range()?
- Why does range(1, 5) not print 5?
- What is an infinite loop?
- Why is indentation important in Python loops?
- Can a for loop work with strings and lists?
- When should we prefer while loop over for loop?

15) Short seminar summary

Loops are used to **repeat** a block of code. Python mainly provides **for loop** and **while loop**. A **for loop** is best when we know how many times to repeat or when we iterate over a sequence. A **while loop** is best when repetition depends on a condition. While using loops, we must remember **indentation**, correct **range()** usage, and proper **variable update** in while loops.

16) Quick revision box

Topic	Key point
Loop	Repeats a block of code multiple times
for loop	Used with sequence / known repetitions
while loop	Runs while condition is True
range()	Generates sequence of numbers
Infinite loop	Loop that never stops because condition never becomes False
Indentation	Mandatory in Python to define loop body